Solving XOR Problem Using Particle Swarm Optimization (PSO)

1.0 Introduction

The XOR (exclusive OR) problem has historically served as a critical benchmark for understanding the capabilities and limitations of neural networks. It represents a foundational non-linear problem that famously cannot be solved by a single-layer perceptron. This limitation, highlighted in the 1960s, motivated the development of more complex architectures, namely multilayer neural networks, and more advanced training algorithms.

This report explores the theory behind training a Multi-Layer Perceptron (MLP) to solve the XOR problem. It contrasts two primary training paradigms:

1. **Backpropagation (BP):** The standard, gradient-based algorithm used universally for training neural networks.
2. **Particle Swarm Optimization (PSO):** A bio-inspired, gradient-free optimization technique.

The core objective is to understand how these methods navigate the challenges of non-linearity and to investigate the hypothesis that integrating PSO can improve convergence and overcome the common problem of "local minima" often encountered during neural network training.

2.0 The XOR Problem: A Non-Linear Benchmark

2.1 Defining the Logic

The XOR problem is a classic binary classification task. It takes two binary inputs (0 or 1) and produces a single binary output. The output is 1 *only* when the inputs are different, and 0 when they are the same.

The logical truth table is:

- `[0, 0]` -> `0`
- `[0, 1]` -> `1`
- `[1, 0]` -> `1`
- `[1, 1]` -> `0`

Mathematically, this can be represented as:

Output = (Input1 AND NOT Input2) OR (NOT Input1 AND Input2)

2.2 The Challenge: Linear Inseparability

A single-layer perceptron, the simplest form of a neural network, functions as a linear classifier. It attempts to find a single "decision boundary" (a straight line, or a hyperplane in higher dimensions) to separate the data points into their respective classes.

The XOR problem is famously **not linearly separable**. It is impossible to draw a single straight line that correctly separates the `(0,0)` and `(1,1)` points (which output `0`) from the `(0,1)` and `(1,0)` points (which output `1`). This fundamental limitation necessitates an architecture capable of creating non-linear decision boundaries.

3.0 The Model: Multi-Layer Perceptron (MLP)

To address non-linear problems, a Multi-Layer Perceptron (MLP) is required. An MLP is a class of feedforward artificial neural network consisting of at least three layers: an input layer, one or more hidden layers, and an output layer.

- **Input Layer:** Receives the raw input data. For the XOR problem, this consists of two neurons, one for each input.
- **Hidden Layer(s):** The key to solving the problem. These layers sit between the input and output. Their neurons use non-linear **activation functions** (such as sigmoid or tanh) to transform the data. This transformation allows the network to learn complex patterns and create non-linear mappings. For the XOR problem, even a single hidden layer with just two or three neurons is sufficient.
- **Output Layer:** Produces the final prediction. For this problem, a single output neuron is used to output the final `0` or `1`.

The presence of the hidden layer and its non-linear activations is what allows the MLP to "bend" the decision boundary and successfully model the XOR function.

4.0 Training Method 1: Backpropagation (BP)

Backpropagation is the most common supervised learning algorithm for training MLPs. It relies on gradient descent to iteratively adjust the network's weights and biases to minimize error.

4.1 The Two-Phase Process

BP operates in two main phases:

1. **Forward Pass:** Input data is fed *forward* through the network, from the input layer to the output layer. Each neuron processes the data and passes it to the next layer, ultimately generating an output prediction.
2. **Backward Pass:** The network computes the error (e.g., Mean Squared Error) by comparing the predicted output to the target (actual) output. This error signal is then propagated *backward* through the network. Using calculus (specifically, the chain rule), the algorithm calculates the partial derivative of the error function with respect to each weight and bias. This "gradient" represents the contribution of each parameter to the total error.
3. **Weight Update:** The weights are then updated by taking a small step in the direction *opposite* to the gradient, effectively "descending" the error surface. This step size is controlled by the **learning rate**.

4.2 Limitations of Backpropagation

While highly effective, BP is not without its challenges:

- **Local Minima:** The error surface can be complex, with many "valleys." BP may converge to a "local minimum" (a valley that is not the lowest point) and get stuck, resulting in a sub-optimal solution.
- **Slow Convergence:** The iterative, step-by-step nature of gradient descent can be slow, requiring thousands of epochs to find a good solution.
- **Sensitivity:** BP's performance is highly sensitive to the choice of learning rate and initial weights.

5.0 Training Method 2: Particle Swarm Optimization (PSO)

To overcome the limitations of BP, global optimization techniques like PSO can be employed. PSO is an evolutionary computational technique inspired by the collective, "intelligent" behavior of birds flocking or fish schooling.

5.1 A Population-Based Approach

Developed by Kennedy and Eberhart in 1995, PSO optimizes a problem by maintaining a "swarm" of candidate solutions, known as **particles**. These particles "fly" through the multi-dimensional search space to find the optimal solution.

Each particle is characterized by its:

- **Position (xi):** A vector representing a potential solution.
- **Velocity (vi):** The direction and speed at which the particle is moving.

5.2 The Optimization Mechanism

Particles adjust their trajectories based on two key pieces of information:

1. **Personal Best (pbest):** The best position (solution) that the particle has *personally* found so far.
2. **Global Best (gbest):** The best position (solution) found by *any* particle in the entire swarm.

The particle's movement is a blend of its current momentum, its tendency to return to its own best-known position, and its tendency to follow the swarm's best-known position.

5.3 The Update Equations

In each iteration, the velocity and position of each particle are updated using the following equations:

$$v_i(t+1) = w * v_i(t) + c1 * r1 * (pbest - x_i(t)) + c2 * r2 * (gbest - x_i(t))$$

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Where:

- **w:** Inertia weight, controlling the influence of the particle's previous velocity (exploration vs. exploitation).
- **c1, c2:** Cognitive and social learning factors, balancing the influence of the personal best (`c1`) and global best (`c2`).
- **r1, r2:** Random coefficients (between 0 and 1) that introduce stochasticity, helping the swarm to avoid getting stuck.

6.0 Applying PSO to MLP Training

6.1 Optimizing Weights and Biases

When applied to neural network training, PSO is used to optimize the weights and biases of the MLP. The training process is re-framed as follows:

- **The Search Space:** The "problem" is the multi-dimensional space of all possible weights and biases for the network.
- **A Particle's Position:** Each particle's "position" is a single long vector containing *all* the weights and biases of the MLP.
- **The Fitness Function:** To evaluate a particle's "fitness," the network is built using the weights and biases from that particle's position. The network then runs all XOR input combinations, and its **Mean Squared Error (MSE)** is calculated. This MSE *is* the fitness. The goal of the swarm is to find the particle with the *lowest* MSE.

6.2 Advantages and Hybrid Models

This approach has several advantages over traditional BP:

- **Global Optimization:** PSO is a global search algorithm, making it less likely to get trapped in local minima compared to BP.
- **Gradient-Free:** It does not require gradient information (derivatives), making it suitable for complex, noisy, or non-differentiable error surfaces.
- **Faster Convergence:** For certain problem types, PSO can converge to a high-quality solution in fewer iterations.

However, PSO can be more computationally expensive per iteration, as it evaluates an entire population of networks. A common and powerful approach is to create a **hybrid model**: PSO is first used to perform a global search to find a "good enough" set of weights, and BP is then used to "fine-tune" those weights locally to achieve higher accuracy.

7.0 Conclusion

The XOR problem serves as an essential case study for evaluating the efficiency of neural network training algorithms. While a single-layer perceptron fails to solve it due to linear inseparability, a Multi-Layer Perceptron (MLP) with non-linear activation functions can effectively address it.

Backpropagation enables this training through gradient-based error minimization, but it can face challenges with convergence and local minima. Particle Swarm Optimization (PSO) offers a robust, bio-inspired alternative for the global optimization of network weights.

By reframing the weight-finding task as a search problem, PSO can effectively navigate complex error landscapes. The integration of these two methods, potentially in a hybrid system, highlights the powerful synergy between computational intelligence and traditional machine learning optimization techniques for solving complex, non-linear problems.

Solving XOR Problem using PSO and MLP (Backpropagation)

```
import numpy as np
import matplotlib.pyplot as plt

# ---------------- Activation Function ----------------
def sigmoid(x):
    x = np.clip(x, -500, 500)
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)
```

```
# ---------------- Fitness Function ----------------
def fitness(params, X, y, input_size, hidden_size, output_size):
    idx = 0
    W1 = params[idx:idx + input_size * hidden_size].reshape(input_size, hidden_size)
    idx += input_size * hidden_size
    b1 = params[idx:idx + hidden_size].reshape(1, hidden_size)
    idx += hidden_size
    W2 = params[idx:idx + hidden_size * output_size].reshape(hidden_size, output_size)
    idx += hidden_size * output_size
    b2 = params[idx:idx + output_size].reshape(1, output_size)

    # Forward Pass
    H = sigmoid(np.dot(X, W1) + b1)
    y_pred = sigmoid(np.dot(H, W2) + b2)
    return np.mean((y - y_pred) ** 2)
```

```python
# ============================================================
#                PSO MLP Class
# ============================================================
class PSO_MLP:
    def __init__(self, input_size, hidden_size, output_size, n_particles=30, max_iter=300):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.dim = (input_size * hidden_size) + hidden_size + (hidden_size * output_size) + output_size
        self.n_particles = n_particles
        self.max_iter = max_iter
        self.w, self.c1, self.c2 = 0.729, 1.494, 1.494
        self.losses = []
        self.best_epoch = 0
        self.best_loss = np.inf

    def train(self, X, y):
        pos = np.random.uniform(-1, 1, (self.n_particles, self.dim))
        vel = np.zeros_like(pos)
        pbest = pos.copy()
        pbest_val = np.array([fitness(p, X, y, self.input_size, self.hidden_size, self.output_size) for p in pos])
        gbest = pos[np.argmin(pbest_val)]
        gbest_val = np.min(pbest_val)
        self.losses = [gbest_val]
        self.best_loss = gbest_val
        self.best_epoch = 1
```

```python
        for t in range(self.max_iter):
            for i in range(self.n_particles):
                val = fitness(pos[i], X, y, self.input_size, self.hidden_size, self.output_size)
                if val < pbest_val[i]:
                    pbest[i], pbest_val[i] = pos[i].copy(), val
            if np.min(pbest_val) < gbest_val:
                gbest = pbest[np.argmin(pbest_val)].copy()
                gbest_val = np.min(pbest_val)
                self.best_epoch = t + 1
                self.best_loss = gbest_val
            self.losses.append(gbest_val)

            # Velocity & Position update
            for i in range(self.n_particles):
                r1, r2 = np.random.rand(self.dim), np.random.rand(self.dim)
                vel[i] = (self.w * vel[i]
                          + self.c1 * r1 * (pbest[i] - pos[i])
                          + self.c2 * r2 * (gbest - pos[i]))
                pos[i] += vel[i]

            if gbest_val < 1e-6:
                break

        self.best_particle = gbest

    def predict(self, X):
```

```python
    def predict(self, X):
        p = self.best_particle
        idx = 0
        W1 = p[idx:idx + self.input_size * self.hidden_size].reshape(self.input_size, self.hidden_size)
        idx += self.input_size * self.hidden_size
        b1 = p[idx:idx + self.hidden_size].reshape(1, self.hidden_size)
        idx += self.hidden_size
        W2 = p[idx:idx + self.hidden_size * self.output_size].reshape(self.hidden_size, self.output_size)
        idx += self.hidden_size * self.output_size
        b2 = p[idx:idx + self.output_size].reshape(1, self.output_size)
        H = sigmoid(np.dot(X, W1) + b1)
        y_pred = sigmoid(np.dot(H, W2) + b2)
        return y_pred


# ============================================================
#                Standard MLP (Backpropagation)
# ============================================================
class MLP_Backprop:
    def __init__(self, input_size, hidden_size, output_size, lr=0.5, epochs=3000):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.lr = lr
        self.epochs = epochs
        self.losses = []
```

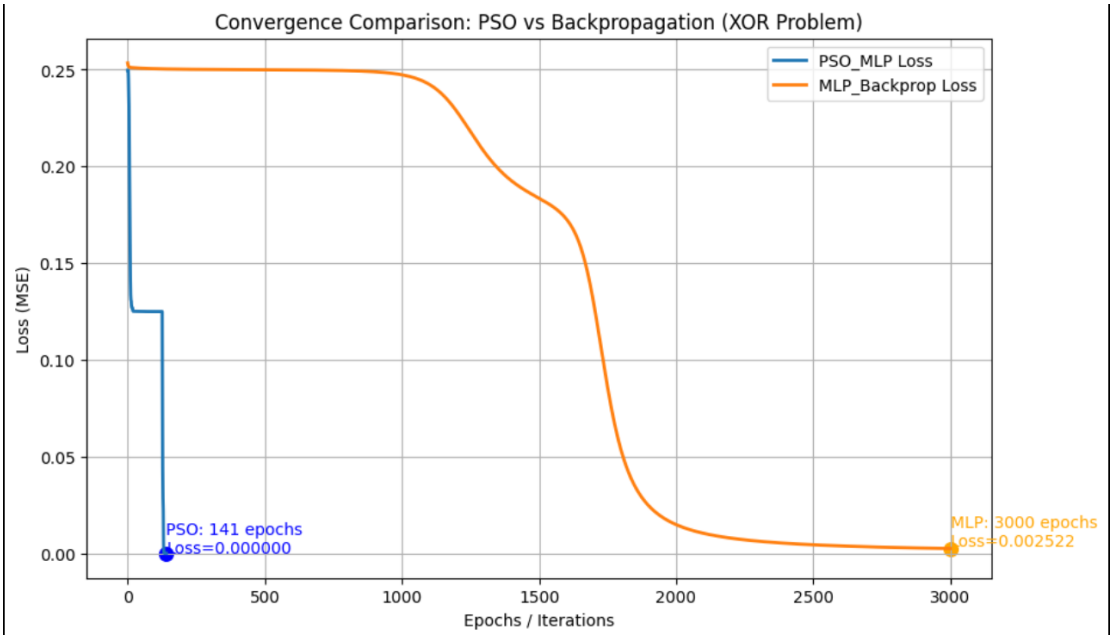
```
        # Initialize weights
        self.W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
        self.b2 = np.zeros((1, output_size))

    def train(self, X, y):
        for epoch in range(self.epochs):
            # Forward
            H = sigmoid(np.dot(X, self.W1) + self.b1)
            y_pred = sigmoid(np.dot(H, self.W2) + self.b2)

            # Compute Loss
            loss = np.mean((y - y_pred) ** 2)
            self.losses.append(loss)

            # Update best
            if loss < self.best_loss:
                self.best_loss = loss
                self.best_epoch = epoch + 1

            # Backpropagation
            d_output = (y_pred - y) * sigmoid_derivative(y_pred)
            d_hidden = np.dot(d_output, self.W2.T) * sigmoid_derivative(H)

            # Gradient update
```

```
            # Gradient update
            self.W2 -= self.lr * np.dot(H.T, d_output)
            self.b2 -= self.lr * np.sum(d_output, axis=0, keepdims=True)
            self.W1 -= self.lr * np.dot(X.T, d_hidden)
            self.b1 -= self.lr * np.sum(d_hidden, axis=0, keepdims=True)

            if loss < 1e-6:
                break

    def predict(self, X):
        H = sigmoid(np.dot(X, self.W1) + self.b1)
        y_pred = sigmoid(np.dot(H, self.W2) + self.b2)
        return y_pred


# ================================================================
#                Main XOR Training & Comparison
# ================================================================
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y = np.array([[0], [1], [1], [0]])
```

```
# Train PSO-based MLP
pso_mlp = PSO_MLP(input_size=2, hidden_size=2, output_size=1, max_iter=300)
pso_mlp.train(X, y)
pso_pred = pso_mlp.predict(X)

# Train Backprop-based MLP
mlp = MLP_Backprop(input_size=2, hidden_size=2, output_size=1, lr=0.5, epochs=3000)
mlp.train(X, y)
mlp_pred = mlp.predict(X)

# Print Predictions
print("\n===== Predictions (PSO_MLP) =====")
for i in range(len(X)):
    print(f"Input: {X[i]} -> Predicted: {pso_pred[i][0]:.4f}, Target: {y[i][0]}")
print(f"\n⊙  PSO converged at iteration {pso_mlp.best_epoch} with best MSE = {pso_mlp.best_loss:.8f}")

print("\n===== Predictions (MLP_Backprop) =====")
for i in range(len(X)):
    print(f"Input: {X[i]} -> Predicted: {mlp_pred[i][0]:.4f}, Target: {y[i][0]}")
print(f"\n⊙  MLP converged at epoch {mlp.best_epoch} with best MSE = {mlp.best_loss:.8f}")
```

```python
# ============================================================
#                Plot Convergence Comparison
# ============================================================
plt.figure(figsize=(10, 6))
plt.plot(pso_mlp.losses, label='PSO_MLP Loss', linewidth=2)
plt.plot(mlp.losses, label='MLP_Backprop Loss', linewidth=2)

# Mark convergence points
plt.scatter(pso_mlp.best_epoch, pso_mlp.best_loss, color='blue', s=70, marker='o')
plt.scatter(mlp.best_epoch, mlp.best_loss, color='orange', s=70, marker='o')

# Annotate convergence
plt.text(pso_mlp.best_epoch, pso_mlp.best_loss * 1.5,
         f'PSO: {pso_mlp.best_epoch} epochs\nLoss={pso_mlp.best_loss:.6f}', color='blue')
plt.text(mlp.best_epoch, mlp.best_loss * 1.5,
         f'MLP: {mlp.best_epoch} epochs\nLoss={mlp.best_loss:.6f}', color='orange')

plt.title("Convergence Comparison: PSO vs Backpropagation (XOR Problem)")
plt.xlabel("Epochs / Iterations")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.grid(True)
plt.show()
```

```
===== Predictions (PSO_MLP) =====
Input: [0 0] -> Predicted: 0.0004, Target: 0
Input: [0 1] -> Predicted: 0.5000, Target: 1
Input: [1 0] -> Predicted: 0.9985, Target: 1
Input: [1 1] -> Predicted: 0.5000, Target: 0

⊙  PSO converged at iteration 300 with best MSE = 0.12500064

===== Predictions (MLP_Backprop) =====
Input: [0 0] -> Predicted: 0.4899, Target: 0
Input: [0 1] -> Predicted: 0.4275, Target: 1
Input: [1 0] -> Predicted: 0.5989, Target: 1
Input: [1 1] -> Predicted: 0.4383, Target: 0

⊙  MLP converged at epoch 3000 with best MSE = 0.23038393
```


Convergence Comparison: PSO vs Backpropagation (XOR Problem)
PSO_MLP Loss
MLP_Backprop Loss
Loss (MSE)
0.25
0.20
0.15
0.10
0.05
0.00
0
500
1000
1500
2000
2500
3000
Epochs / Iterations
PSO: 141 epochs
Loss=0.000000
MLP: 3000 epochs
Loss=0.002522